COMPUTER ENGINEERING

QUEZON CITY

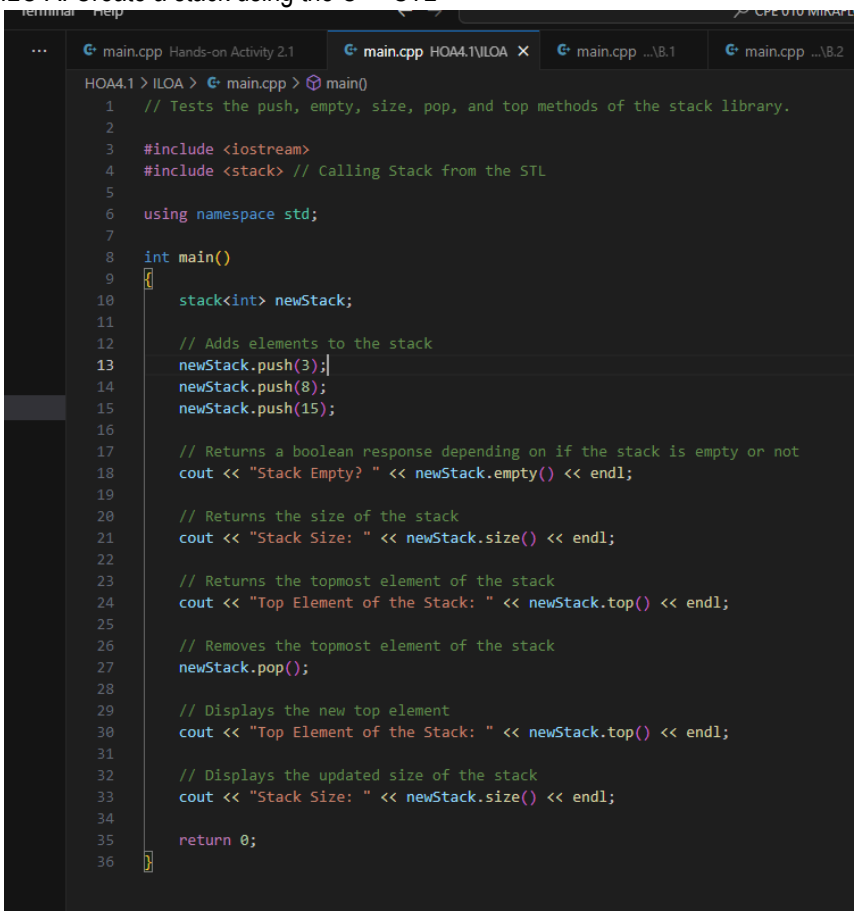
COURSE CODE/TITLE:	CPE 0010	**SECTION:**	CPE21S4
DATE PERFORMED:	31/07/2026	**DATE SUBMITTED:**	31/07/2026
NAME:	Seth Carlo T. Miraflor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create a stack using the C++ STL



ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

B.1. Stacks using Arrays

main.cpp Hands-on Activity 2.1 | main.cpp HOA4.1\ILOA | main.cpp ...\B.1 | main.cpp ...\B.2 | main.cpp ...\A | main.cpp ...\B

HOA4.1 > ILOAB > B.1 > main.cpp > main()

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap]; // Stack with max of 100 elements
int top = -1, i, newData;

void push();
void pop();
void Top();
void display();
bool isEmpty();

int main() {
    int choice;

    std::cout << "Enter number of max elements for new stack: ";
    std::cin >> i;

    while (true) {
        std::cout << "Stack Operations: " << std::endl;
        std::cout << "1. PUSH, 2. POP, 3. TOP, 4. isEMPTY, 5. DISPLAY" << std::endl;
        std::cin >> choice;

        switch (choice) {
        case 1:
            push();
            break;
        case 2:
            pop();
            break;
        case 3:
            Top();
            break;
        case 4:
            std::cout << isEmpty() << std::endl;
            break;
        case 5:
            display();
            break;
        default:
            std::cout << "Invalid Choice." << std::endl;
            break;
        }
    }

    return 0;
}
```

HOA4.1 > ILOAB > B.1 > main.cpp > main()

```cpp
bool isEmpty() {
    if (top == -1)
        return true;
    return false;
}

void push() {
    // Check if full
    if (top == i - 1) {
        std::cout << "Stack Overflow." << std::endl;
        return;
    }

    std::cout << "New Value: ";
    std::cin >> newData;
    stack[++top] = newData;
}

void pop() {
    // Check if empty
    if (isEmpty()) {
        std::cout << "Stack Underflow." << std::endl;
        return;
    }

    // Display the top value
    std::cout << "Popping: " << stack[top] << std::endl;

    // Decrement top
    top--;
}

void Top() {
    if (isEmpty()) {
        std::cout << "Stack is Empty." << std::endl;
        return;
    }

    std::cout << "The element on the top of the stack is "
              << stack[top] << std::endl;
}

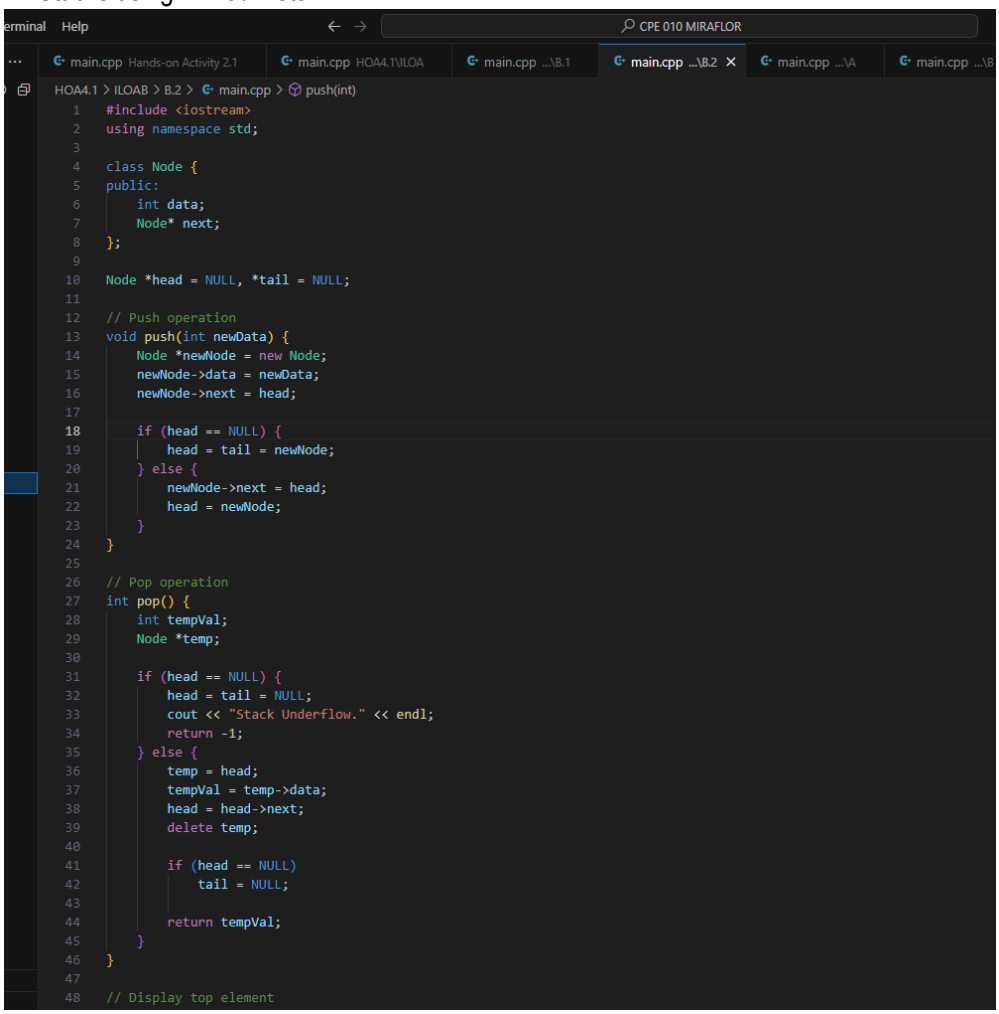
void display() {
    // Check if empty
    if (isEmpty()) {
        std::cout << "Stack is Empty." << std::endl;
        return;
    }
```

```cpp
    std::cout << "Stack Elements: ";

    for (int j = top; j >= 0; j--) {
        std::cout << stack[j] << " ";
    }

    std::cout << std::endl;
}
```

B.2. Stacks using Linked Lists




```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
};

Node *head = NULL, *tail = NULL;

// Push operation
void push(int newData) {
    Node *newNode = new Node;
    newNode->data = newData;
    newNode->next = head;

    if (head == NULL) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head = newNode;
    }
}

// Pop operation
int pop() {
    int tempVal;
    Node *temp;

    if (head == NULL) {
        head = tail = NULL;
        cout << "Stack Underflow." << endl;
        return -1;
    } else {
        temp = head;
        tempVal = temp->data;
        head = head->next;
        delete temp;

        if (head == NULL)
            tail = NULL;

        return tempVal;
    }
}

// Display top element
```

```
47
48  // Display top element
49  void Top() {
50      if (head == NULL) {
51          cout << "Stack is Empty." << endl;
52      } else {
53          cout << "Top of Stack: " << head->data << endl;
54      }
55  }
56
57  // Display all elements
58  void Display() {
59      if (head == NULL) {
60          cout << "Stack is Empty." << endl;
61          return;
62      }
63
64      Node *temp = head;
65
66      cout << "Stack Elements (Top to Bottom): ";
67
68      while (temp != NULL) {
69          cout << temp->data << " ";
70          temp = temp->next;
71      }
72
73      cout << endl;
74  }
75
76  int main() {
77
78      push(1);
79      cout << "After the first PUSH:" << endl;
80      Top();
81      Display();
82
83      push(5);
84      cout << "\nAfter the second PUSH:" << endl;
85      Top();
86      Display();
87
88      pop();
89      cout << "\nAfter the first POP:" << endl;
90      Top();
91      Display();
92
93      pop();
```

```
93      pop();
94      cout << "\nAfter the second POP:" << endl;
95      Top();
96      Display();
97
98      pop();
99
100     return 0;
101 }
```

Analysis

- the internal representation of a stack affects its implementation, memory usage, and flexibility. A stack using an array is simple and fast, with **O(1)** push and pop operations, but it has a fixed size that can cause stack overflow if it becomes full. A stack using a linked list is more flexible because it can grow dynamically as needed, although it requires extra memory for pointers. Meanwhile, the C++ STL stack is the easiest to use since it is already built into the language, making the code simpler, more reliable, and efficient. Overall, arrays are best for fixed-size stacks, linked lists are suitable for dynamic memory needs, and the C++ STL is the most convenient choice for general programming

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
ILO C: Solve problems using an implementation of stack:

A.




```cpp
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

class Stack {
private:
    char arr[MAX];
    int top;

public:
    Stack() {
        top = -1;
    }

    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == MAX - 1;
    }

    void push(char ch) {
        if (!isFull())
            arr[++top] = ch;
    }

    char pop() {
        if (!isEmpty())
            return arr[top--];
        return '\0';
    }
};

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalance(string expr) {
    Stack s;

    for (char ch : expr) {

        if (ch == '(' || ch == '{' || ch == '[')
```

```
48          if (ch == '(' || ch == '{' || ch == '[')
49              s.push(ch);
50
51          else if (ch == ')' || ch == '}' || ch == ']') {
52
53              if (s.isEmpty())
54                  return false;
55
56              char top = s.pop();
57
58              if (!isMatching(top, ch))
59                  return false;
60          }
61      }
62
63      return s.isEmpty();
64  }
65
66  int main() {
67
68      string expr;
69
70      cout << "Enter expression: ";
71      getline(cin, expr);
72
73      if (checkBalance(expr))
74          cout << "Expression is Balanced." << endl;
75      else
76          cout << "Expression is NOT Balanced." << endl;
77
78      return 0;
79  }
```

B.

HOA4.1 > ILOC > B > main.cpp > checkBalance(string)

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Node {
    char data;
    Node* next;
};

class Stack {
private:
    Node* top;

public:
    Stack() {
        top = NULL;
    }

    bool isEmpty() {
        return top == NULL;
    }

    void push(char ch) {
        Node* temp = new Node;
        temp->data = ch;
        temp->next = top;
        top = temp;
    }

    char pop() {

        if (isEmpty())
            return '\0';

        Node* temp = top;
        char value = temp->data;

        top = top->next;
        delete temp;

        return value;
    }
};

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
```

```cpp
}

bool checkBalance(string expr) {

    Stack s;

    for (char ch : expr) {

        if (ch == '(' || ch == '{' || ch == '[')
            s.push(ch);

        else if (ch == ')' || ch == '}' || ch == ']') {

            if (s.isEmpty())
                return false;

            char top = s.pop();

            if (!isMatching(top, ch))
                return false;
        }
    }

    return s.isEmpty();
}

int main() {

    string expr;

    cout << "Enter expression: ";
    getline(cin, expr);

    if (checkBalance(expr))
        cout << "Expression is Balanced." << endl;
    else
        cout << "Expression is NOT Balanced." << endl;

    return 0;
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.